DSA MASTER REVISION GUIDE

Kashyap Pandey

Principal Engineer Prep | Microsoft • Google • Meta • Amazon

v2.0 | April 1, 2026 | Trees → Graphs → DP → Patterns

How To Use This Document

Open before any interview or mock session. Read trick sentence for each pattern. Glance at pseudocode. Never re-read full solutions — trust your practice. This document updates after every session automatically.

- Before interview: trick sentences only — 10 min
- Day before: pseudocode + templates — 30 min
- Week before: cold reproduce 2-3 problems from memory

PATTERN 1 — BINARY TREES — DFS (Traversals)

TRICK: *Inorder=Left→Me→Right (sorted on BST). Preorder=Me first (copy tree). Postorder=Me last (height/delete). Every method calls ITSELF. Base case always first.*

Core Concepts

- Root = top node, no parent
- Leaf = bottom node, no children
- Height = longest path root to leaf
- Every node IS a tree — recursion is natural language of trees
- DFS = go deep first (Inorder/Preorder/Postorder) — uses Stack/Recursion
- BFS = go wide first (Level Order) — uses Queue

DFS vs BFS — When To Use

DFS — Use When	BFS — Use When
Path problems (root to leaf)	Level by level processing
Height, diameter	Shortest path (unweighted)

BST operations	Right/left side view
Subtree result needed first	Problem mentions 'level' or 'layer'
Copy/serialize tree	Connect nodes at same level

The Universal Tree Recursion Template

```
void traverse(TreeNode node) {
    if (node == null) return;    // BASE CASE — always first
    // [preorder work]           // BEFORE = root processed first
    traverse(node.left);         // go left
    // [inorder work]            // BETWEEN = root processed middle
    traverse(node.right);        // go right
    // [postorder work]          // AFTER = root processed last
}
```

Problems Solved

#	Problem	Difficulty	Score
1	Binary Tree Inorder Traversal (Recursive) LC#94	Easy	☑ 4/5
2	Binary Tree Inorder Traversal (Iterative) LC#94	Easy	☑ 3.5/5
3	Binary Tree Level Order Traversal BFS LC#102	Medium	☑ 5/5 ✸
4	Preorder Traversal Iterative LC#144	Easy	☑ 5/5 ✸
5	Postorder Traversal Iterative LC#145	Easy	☑ 5/5 ✸
6	Maximum Depth of Binary Tree LC#104	Easy	☑ 5/5 ✸
7	Diameter of Binary Tree LC#543	Easy	☑ 4/5
8	Invert Binary Tree LC#226	Easy	☑ 5/5 ✸
9	Symmetric Tree LC#101	Easy	☑ 4/5

Pseudocode — Preorder Iterative

```
stack.push(root)
while stack not empty:
    node = stack.pop()
    result.add(node.val)              // process IMMEDIATELY
    if node.right!=null: stack.push(node.right)  // right FIRST
    if node.left!=null:  stack.push(node.left)   // left SECOND
                                      // LIFO → left processed first ☑
```

Pseudocode — Postorder Iterative

```
// Trick: reverse of (Root→Right→Left) = (Left→Right→Root) = Postorder
```

```
stack.push(root)
while stack not empty:
    node = stack.pop()
    result.add(node.val)
    if node.left!=null:  stack.push(node.left)   // left FIRST (swapped)
    if node.right!=null: stack.push(node.right)  // right SECOND (swapped)
Collections.reverse(result)           // reverse at end → postorder ☑
```

Pseudocode — Maximum Depth (LC#104)

```
// 'My depth = 1 + deeper of my two subtrees. Base: null=0.'
int maxDepth(TreeNode root):
    if root==null: return 0
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right))

// Postorder: children report first, parent computes after
// Time: O(n)   Space: O(h)
```

Pseudocode — Diameter of Binary Tree (LC#543)

```
// 'At every node: diameter = leftH+rightH. Return HEIGHT not diameter.'
int diameter = 0;   // global tracker

int height(TreeNode node):
    if node==null: return 0
    int leftH  = height(node.left)
    int rightH = height(node.right)
    diameter = Math.max(diameter, leftH+rightH)  // UPDATE global
    return 1 + Math.max(leftH, rightH)           // RETURN height (not diameter!)

// CRITICAL BUG: return 1+diameter is WRONG — corrupts height calculation
// diameter = global (grows across nodes)
// height   = local  (specific to this subtree)
// NEVER mix them in the return statement
```

Pseudocode — Invert Binary Tree (LC#226)

```
// 'At every node: swap left and right. Recurse both. One swap per node.'
TreeNode invertTree(TreeNode root):
    if root==null: return null
    TreeNode temp = root.left       // save
    root.left     = root.right      // swap
    root.right    = temp            // swap
    invertTree(root.left)           // recurse
    invertTree(root.right)          // recurse
    return root

// Preorder: swap FIRST, recurse after — parent independent of children
// Postorder also works — swap doesn't depend on children's results
```

Pseudocode — Symmetric Tree (LC#101)

```
// 'Compare TWO nodes at once — one from each side.'
```

```
boolean isSymmetric(root): return isMirror(root.left, root.right)

boolean isMirror(left, right):
    if left==null && right==null: return true    // both null → ok
    if left==null || right==null: return false   // one null → fail
    if left.val != right.val:     return false   // mismatch → fail
    return isMirror(left.left,  right.right)     // outer pair
        && isMirror(left.right, right.left)      // inner pair

// CRITICAL BUG: if(val==val) return true → skips children check
// CORRECT:      if(val!=val) return false → only stops on mismatch
// Matching values = necessary but NOT sufficient → always recurse
```

Pseudocode — Inorder Traversal (Recursive)

```
List<Integer> result = new ArrayList<>();

void helper(TreeNode node, List result):
    if node==null: return              // base case
    helper(node.left, result)          // LEFT
    result.add(node.val)               // ROOT
    helper(node.right, result)         // RIGHT
```

Pseudocode — Inorder Traversal (Iterative)

```
Deque<TreeNode> stack = new ArrayDeque<>();
TreeNode curr = root;

while curr!=null OR stack not empty:
    // Step 1: go LEFT as far as possible
    while curr!=null:
        stack.push(curr)
        curr = curr.left

    // Step 2: pop ONE node → add → go right
    curr = stack.pop()          // pop ONE (not while loop!)
    result.add(curr.val)
    curr = curr.right           // outer loop handles left

// KEY: pop is never a while loop — pops ONE, then outer loop
//      goes left again from the right subtree
```

Pseudocode — Level Order BFS

```
if root==null: return []
Queue<TreeNode> queue = new LinkedList<>();
queue.offer(root);                     // seed

while queue not empty:
    int levelSize = queue.size()       // SNAPSHOT — key trick
    List<Integer> level = new ArrayList<>()

    for i=0 to levelSize-1:
        node = queue.poll()            // remove from front
```

```
        level.add(node.val)
        if node.left!=null:  queue.offer(node.left)
        if node.right!=null: queue.offer(node.right)

    result.add(level)

// TRICK: snapshot size BEFORE loop = exactly one level
```

Complexity — All Traversals

Algorithm	Time	Space	Space Worst Case
Inorder Recursive	**O(n)**	**O(h)**	O(n) skewed tree
Inorder Iterative	**O(n)**	**O(h)**	O(n) skewed tree
Preorder Recursive	**O(n)**	**O(h)**	O(n) skewed tree
Postorder Recursive	**O(n)**	**O(h)**	O(n) skewed tree
Level Order BFS	**O(n)**	**O(w)**	O(n) last level = n/2

PATTERNS 2–12 ADDED AS WE COVER THEM

Coming next: Trees Advanced (Apr 2-6) → BST (Apr 14) → Graphs (Apr 21) → Backtracking (May 1) → DP (May 5) → Heaps (May 19) → LLD (May 26) → Core Patterns (Jun 1)

Progress Tracker

Pattern	Solved	Target	Status	Last Updated
Binary Trees — Traversals	**9**	12	🟡 In Progress	Apr 1-3
Binary Trees — Advanced	**0**	10	☐ Pending	—
BST	**0**	8	☐ Pending	—
Graphs — BFS/DFS	**0**	10	☐ Pending	—
Graphs — Advanced	**0**	6	☐ Pending	—
Backtracking	**0**	8	☐ Pending	—

DP — 1D	**0**	10	☐ Pending	—
DP — 2D	**0**	8	☐ Pending	—
Heaps	**0**	8	☐ Pending	—
LLD — Design Patterns	**0**	6	☐ Pending	—
Sliding Window	**0**	8	☐ Pending	—
Two Pointers	**0**	8	☐ Pending	—
Binary Search	**0**	7	☐ Pending	—
Stack / Monotonic	**0**	7	☐ Pending	—
Intervals	**0**	6	☐ Pending	—
Tries	**0**	5	☐ Pending	—

v2.2 | Last Updated: Apr 3, 2026 | Total Solved: 9 / 127 | Target: Jun 30, 2026

"Depth beats breadth. Every time."